Progress Report

Intelligent Distributed Compute Fabric for Heterogeneous Personal Devices

15–24 September 2026

Rishu Raj (23UCS085) · Abhay Singh (23UCS097) · Aayush Kumar (23UCS095) · Aditya Kumar Tiwari (23UCS099)

1 WHERE WE ARE NOW

3	2,600	13	149
VERSIONS BUILT	AUTOMATIC TESTS ON THE MAIN SYSTEM	DESIGN DECISIONS RECORDED	CODE COMMITS

- The full plan is written version by version, and the design is fixed and agreed.
- Our laptops now find each other over the network and stay connected.
- A laptop joins, reports its hardware, keeps reporting how busy it is, and reconnects by itself if the network drops.
- Everything else in the project is built on top of this.

2 PROGRESS SINCE THE FIRST REPORT

STAGE	WHAT IT COVERS	STATUS
Base setup	Code structure, database, logging, test setup, and an automatic check that runs every test on Windows and Linux after each change.	Done
Testing the risky ideas	Seven small experiments: finding laptops on the network, Windows–Linux communication, data transfer speed, keeping programs isolated, and using the graphics card (GPU) inside a container.	Done
Core system	The main laptop and the background programs on the others — joining, identity, staying alive, hardware detection, load reporting.	Mostly done
Sending a task away	Send a complete task to another laptop, run it there, bring the result back.	Next

What works today:

- Every laptop creates its own identity on first start, so the main laptop knows which machine is which.
- A laptop can join, and it is handled correctly even if the same laptop connects twice.
- The main laptop knows within a few seconds if a machine is switched off or disconnected.
- Each laptop detects its CPU, memory, graphics card and disk and keeps reporting its load. This is saved, because our scheduling program will learn from it later.
- If the main laptop restarts, the others come back online in one to two seconds by themselves.
- Being finished now: a command that lists all connected machines, and starting the background program automatically when a laptop is switched on.

3 ONE CHANGE IN OUR PLAN

The first report listed four ways of running work. After estimating the real work involved, we decided to complete two of them properly instead of leaving all four half finished.

WAY OF RUNNING WORK	WHAT IT DOES	IN THIS PROJECT
Send the whole task	The complete task goes to one stronger laptop; only the result comes back.	Will be completed
Split the task	The task is cut into parts, every laptop runs a part, the results are joined.	Will be completed
Use another laptop's graphics card	The program stays here; its graphics calculations run on another machine's card.	Designed, added later
Join several graphics cards	Several graphics cards work together as one for a single large AI task.	Designed, added later

- Both of the later two are fully designed and written down.
- The system is built so they can be added without changing the design.
- Choosing where to run the work, measuring everything, and the machine learning that predicts how long a task will take all remain in the plan as before.

4 OUR APPROACH TO EACH HARD PART

THE PROBLEM	WHAT WE DO	WHY THIS WORKS
Connecting the machines	• Each laptop runs a small background program. • It calls the main laptop itself and keeps one line open. • It sends an “I am alive” message every 2 seconds.	• On normal Wi-Fi a laptop cannot receive a call from outside, so it has to call out. • If the line drops it reconnects on its own and says what it is still running.
Sending the code and data	• Every file gets a short fingerprint made from its own contents. • We ask the other laptop which fingerprints it already has. • Only the missing ones are sent.	• The same fingerprint means the same file, so a dataset is downloaded only once. • Files go laptop to laptop, so the main laptop is never the bottleneck.
Running it safely there	• The task runs inside a sealed box on that laptop. • The box gets its own limited memory and processor share. • It cannot touch the rest of that machine.	• We are running somebody else's code, so it has to be boxed in. • The box is reused, so it is prepared only once per machine.
Dividing work between unequal laptops	• The work is cut into many small pieces in one shared list. • Each laptop takes a piece when it is free, finishes it, takes the next.	• Nobody has to calculate shares — a fast laptop simply takes more pieces. • The laptop with no graphics card still helps, with a smaller share.
A laptop is closed mid-job	• Every piece is taken with a time limit. • If that laptop stops answering, the limit runs out. • The piece returns to the list for someone else.	• The job carries on without being restarted. • A slow last piece is also given to a free machine, and the first result wins.
Deciding where to run	• List the possible ways to run it. • Remove the ones that cannot work. • Estimate the time of each and pick the best.	• Transfer time can be calculated: data size ÷ measured network speed. • Computing time cannot, so it is learned from earlier runs on that machine.
Using another laptop's graphics card	• The numbers are sent over the network. • The calculation happens on the machine that has the card. • Only the result comes back.	• Making a card look as if it were plugged into another laptop is not possible over Wi-Fi. • Working one level higher, inside the AI library, is.

5 WHAT WE HAVE ALREADY BUILT — AND WITH WHAT

PART	HOW IT IS ACTUALLY DONE
Connecting the machines	• **gRPC** — a ready-made way for a program on one computer to call a program on another over the network, so we did not have to invent our own. • **Protocol Buffers** — every message they may send is written down in advance, fixing its exact shape so the two sides can never misunderstand each other. • The line stays open in both directions: a laptop sends a “hello” with its ID and hardware, the main laptop answers. • After that, a short “I am alive” every **2 seconds**. Nothing for **6 seconds** → that laptop is marked offline.
Finding the main laptop	• **mDNS** — the same thing that makes a printer or a speaker appear on your Wi-Fi by itself. • The main laptop announces itself on the network; the others hear it and connect. • Nobody has to type an IP address.
Machine identity	• Each laptop makes two matching keys: a private one it never shows anyone, and a public one it gives out. • Anything checked with the public key proves it came from the machine holding the private one. • **ECDSA P-256** — the same method websites use for the padlock in your browser. • The private key sits in a file the operating system allows only that one user to open. • **ULID** — an ID generated so that two machines can never end up with the same one.
Detecting the hardware	• **psutil** — reads the CPU, memory and disks from the operating system. • **NVML** — NVIDIA's own tool; gives the graphics card's model, its memory and how busy it is. • If NVML is not installed, we run the `nvidia-smi` command instead. • The whole check has a time limit, so a stuck graphics driver cannot stop the laptop from joining.
Reporting the load	• A laptop sends a reading only when something actually changes, instead of sending constantly and filling the network. • The main laptop keeps the latest picture in memory. • Every minute's average and highest value is saved into **PostgreSQL**, the database where all our records are kept.
The main laptop's program	• One Python program runs two servers side by side. • **FastAPI** — answers the ordinary web requests made by the dashboard and the command line. • The gRPC server — handles the always-open connections coming from the other laptops.
Command-line tool	• **Typer** — a library for making command-line tools. • It turns our code into commands a person can type, such as `compute nodes` and `compute status`.

6 DECIDED AND DESIGNED, NOT YET WRITTEN

PART	THE METHOD WE WILL USE
Sending code and data	• **SHA-256 hash** — a short fingerprint worked out from what is inside the file. • The same fingerprint means the same file; change one letter and the fingerprint changes. • We ask the other laptop which fingerprints it has and send only what is missing. • Files travel straight from one laptop to another over ordinary **HTTP**. • The fingerprint part is already written.
Running the task there	• **Docker** — runs a program inside a sealed box with its own share of memory and processor, and no way to touch the rest of the machine. • The graphics card is passed into that box using NVIDIA's container toolkit. • On a laptop without Docker we use **uv** to build a separate Python setup instead.

PART	THE METHOD WE WILL USE
Splitting the work	• All the pieces sit in one shared list. • A laptop takes a piece with a **lease** — simply a time limit on it. • If that laptop stops answering, the time runs out and the piece goes back into the list for someone else. • Nobody has to notice the failure and restart anything.
Deciding where to run	• List every possible way to run the task, throw out the ones that cannot work, estimate how long each would take, pick the best. • Plain rules to begin with, such as choosing the strongest free machine. • Later a model built with **LightGBM / scikit-learn** — standard machine-learning libraries — trained on how long our own earlier runs actually took.
Using another laptop's graphics card	• Done inside **PyTorch**, the AI library the program is already using, instead of trying to fake a graphics card at the driver level. • The model is packed up with PyTorch's own `torch.export`, so we send the model and not the program's code. • The numbers travel over the network, and the model stays sitting in the other laptop's graphics memory so it is not sent again and again.
Security	• Creating the fabric sets up a **certificate authority** — the office that issues and signs ID cards for machines. • A new laptop joins using a one-time code and receives its own certificate. • After that every connection uses **mTLS**: the same thing as HTTPS, except both sides show their ID, not only the server.

7 HOW WE CHECK THAT IT WORKS

- Every part goes through three separate steps — one writes the code, a second writes tests from the requirements and tries to break it, a third reviews the design and correctness and must approve it.
- A task is marked complete only after that, and only if the success conditions written in advance are met.
- Example: in the load-reporting part, the testing step found three serious bugs in the first version — including one where repeated messages after a network drop corrupted the saved readings. All were fixed before it was accepted.

> **Tested so far on one machine and through automatic tests only.** The two-laptop and four-laptop tests on real hardware are the immediate next step, and the real numbers must come from there.

8 WHAT WE DO NEXT

1. Run our hardware script on the other three laptops.
2. Run the two-laptop network tests for real — Wi-Fi and cable, Windows to Linux.
3. Finish the current version: the command that lists machines, and automatic start-up.
4. Send a complete task to another laptop and bring the result back.

9 WHAT WE NEED

- **A network switch and cables** — so we can also measure on a wired connection. Wi-Fi speed changes too much for reliable measurements.
- **Time with all four laptops together** — these tests cannot be done by one person.